

EXERCÍCIO PRÁTICO

Mini catálogo 3D: modelagem, representação e instâncias

Python + Pygame | Computação Gráfica / Realidade Estendida

Aula relacionada	Aula 7 — Modelagem e representação
Produto	Aplicação interativa com cubos, pirâmides e instâncias em uma cena 3D projetada em uma janela 2D
Duração sugerida	2 a 3 horas
Pré-requisitos	Python, Pygame, funções, listas, vetores 3D e noções de projeção perspectiva

1. Contexto e objetivos

Em Computação Gráfica, modelar significa construir uma descrição computável de uma forma. Essa descrição pode ser reutilizada, transformada, selecionada e apresentada como arame, superfície ou sólido. Neste exercício, você construirá um mini catálogo 3D usando duas malhas — cubo e pirâmide — e várias instâncias dessas malhas em posições, escalas, rotações e cores diferentes.

- Representar objetos por vértices e faces, distinguindo geometria e instância.
- Desenhar uma malha como superfície preenchida e como representação em arame.
- Reutilizar uma mesma definição geométrica em várias ocorrências.
- Aplicar translação, rotação e escala a objetos em coordenadas locais.
- Relacionar espaço local, espaço do mundo, câmera e projeção perspectiva.
- Implementar seleção por teclado e feedback visual para o objeto selecionado.

2. Enunciado

Desenvolva uma aplicação chamada “Mini catálogo 3D”. A cena deverá apresentar pelo menos quatro objetos: dois cubos e duas pirâmides. Os objetos devem ser construídos a partir de listas de vértices e faces. A malha do cubo deve ser reutilizada nas duas instâncias, assim como a malha da pirâmide deve ser reutilizada nas duas instâncias.

O usuário poderá alternar a seleção entre os objetos, mover, girar e redimensionar o objeto selecionado, além de deslocar a câmera. A seleção deverá ser indicada por um contorno ou marcador visual. A aplicação deve permitir alternar entre uma visualização de superfície e uma visualização inspirada no modelo de arame.

A implementação deve ser feita sem uma engine 3D pronta. O objetivo é explicitar a passagem por modelagem, representação, transformações, câmera, projeção e desenho na tela.

3. Conceitos mobilizados

Conceito	Aplicação no exercício
Malha poligonal	Vértices descrevem posições; faces conectam vértices e formam a superfície.

Representação	A cena pode ser exibida como superfície preenchida ou como inspeção em arame.
Instanciação	Uma definição de cubo ou pirâmide é usada por várias ocorrências com poses diferentes.
Transformações	Escala local, rotação e translação posicionam cada instância no mundo.
Coordenadas	O modelo nasce no espaço local, é transformado no mundo e convertido para a câmera.
Interação	TAB seleciona; teclado transforma; contorno fornece feedback ao usuário.

4. Preparação do ambiente

Crie uma pasta para a atividade, instale o Pygame e execute o programa.

```
python -m pip install pygame python mini_catalogo_3d.py
```

Em alguns sistemas, use python3 no lugar de python. Salve o código a seguir como mini_catalogo_3d.py. Leia os comentários e identifique as partes que correspondem ao pipeline gráfico.

5. Código-base completo

```
import math
import pygame

pygame.init()

LARGURA, ALTURA = 1100, 720
tela = pygame.display.set_mode((LARGURA, ALTURA))

pygame.display.set_caption("Mini catalogo 3D - Aula 7")
relogio = pygame.time.Clock()
fonte = pygame.font.SysFont("Arial", 18)

# Cada malha e uma lista de vertices e uma lista de faces.
# A mesma malha pode ser reutilizada por varias instancias.
CUBO = [
    (-1,-1,-1), (1,-1,-1), (1,1,-1), (-1,1,-1),
    (-1,-1,1), (1,-1,1), (1,1,1), (-1,1,1)
]
FACES_CUBO = [
    (0,1,2,3), (4,7,6,5), (0,4,5,1),
    (3,2,6,7), (1,5,6,2), (0,3,7,4)
]
PIRAMIDE = [
    (-1,-1,-1), (1,-1,-1), (1,-1,1), (-1,-1,1), (0,1,0)
]
FACES_PIRAMIDE = [
    (0,1,2,3), (0,4,1), (1,4,2), (2,4,3), (3,4,0)
]

def rotacionar(p, rot):
    x, y, z = p
    rx, ry, rz = rot
    c, s = math.cos(rx), math.sin(rx)
    y, z = y*c - z*s, y*s + z*c
    c, s = math.cos(ry), math.sin(ry)
    x, z = x*c + z*s, -x*s + z*c
    c, s = math.cos(rz), math.sin(rz)
    x, y = x*c - y*s, x*s + y*c

    return x, y, z

def transformar(p, instancia):
    escala = instancia["escala"]
    pos = instancia["pos"]
```

```

    p = (p[0]*escala[0], p[1]*escala[1], p[2]*escala[2])
    p = rotacionar(p, instancia["rot"])

    return (p[0]+pos[0], p[1]+pos[1], p[2]+pos[2])

def projetar(p, camera, foco=560):
    x, y, z = p
    x -= camera[0]
    y -= camera[1]
    z -= camera[2]
    if z <= 0.2:
        return None

    return (int((LARGURA/2 + foco*x/z), int(ALTURA/2 - foco*y/z))

def desenhar_grade(superficie, camera):
    for i in range(-8, 9):
        a = projetar((i, -2, 2), camera)
        b = projetar((i, -2, 16), camera)
        c = projetar((-8, -2, i+10), camera)
        d = projetar((8, -2, i+10), camera)
        if a and b: pygame.draw.line(superficie, (55,65,80), a, b, 1)
        if c and d: pygame.draw.line(superficie, (55,65,80), c, d, 1)

def desenhar_malha(superficie, malha, instancia, camera, cor, selecionado=False):
    pontos3d = [transformar(v, instancia) for v in malha["vertices"]]
    pontos2d = [projetar(v, camera) for v in pontos3d]
    for face in malha["faces"]:
        pontos = [pontos2d[i] for i in face]
        if all(pontos):
            pygame.draw.polygon(superficie, cor, pontos)
            pygame.draw.polygon(superficie, (230,235,242), pontos, 1)
    if selecionado:
        validos = [p for p in pontos2d if p]
        if validos:
            xs = [p[0] for p in validos]
            ys = [p[1] for p in validos]
            ret = pygame.Rect(min(xs)-6, min(ys)-6, max(xs)-min(xs)+12, max(ys)-min(ys)+12)
            pygame.draw.rect(superficie, (255,235,80), ret, 2)

malhas = {
    "cubo": {"vertices": CUBO, "faces": FACES_CUBO},
    "piramide": {"vertices": PIRAMIDE, "faces": FACES_PIRAMIDE}
}

def nova_instancia(tipo, pos, escala, rot=(0,0,0), cor=(80,170,245)):
    return {"tipo": tipo, "pos": list(pos), "escala": list(escala), "rot": list(rot), "cor": cor}

objetos = [
    nova_instancia("cubo", (-3, 0, 8), (1.4,1.4,1.4), cor=(65,155,235)),
    nova_instancia("piramide", (0, 0, 9), (1.5,1.5,1.5), cor=(245,165,65)),
    nova_instancia("cubo", (3, 0, 11), (1.0,2.0,1.0), rot=(0.2,0.4,0), cor=(110,215,150)),
    nova_instancia("piramide", (0, 2.7, 10), (0.7,0.7,0.7), cor=(220,105,205)),
]

camera = [0, 1, 0]
selecionado = 0
modo_arame = False
rodando = True

while rodando:
    dt = relogio.tick(60) / 1000
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            rodando = False
        if evento.type == pygame.KEYDOWN:
            if evento.key == pygame.K_TAB:
                selecionado = (selecionado + 1) % len(objetos)
            if evento.key == pygame.K_SPACE:
                modo_arame = not modo_arame
            if evento.key == pygame.K_r:
                camera = [0, 1, 0]
                selecionado = 0
        teclas = pygame.key.get_pressed()
        alvo = objetos[selecionado]
        v = 3 * dt

```

```

# Setas: translacao no plano XZ.

if teclas[pygame.K_LEFT]: alvo["pos"][0] -= v
if teclas[pygame.K_RIGHT]: alvo["pos"][0] += v
if teclas[pygame.K_UP]: alvo["pos"][2] -= v
if teclas[pygame.K_DOWN]: alvo["pos"][2] += v

# A/D: rotacao em Y; W/S: rotacao em X.

if teclas[pygame.K_a]: alvo["rot"][1] -= 1.8*dt
if teclas[pygame.K_d]: alvo["rot"][1] += 1.8*dt
if teclas[pygame.K_w]: alvo["rot"][0] -= 1.8*dt
if teclas[pygame.K_s]: alvo["rot"][0] += 1.8*dt

# Q/E: escala uniforme.

if teclas[pygame.K_q]: alvo["escala"] = [max(0.2,x-1.2*dt) for x in alvo["escala"]]
if teclas[pygame.K_e]: alvo["escala"] = [x+1.2*dt for x in alvo["escala"]]

# J/L e I/K: camera.

if teclas[pygame.K_j]: camera[0] -= v
if teclas[pygame.K_l]: camera[0] += v
if teclas[pygame.K_i]: camera[1] += v
if teclas[pygame.K_k]: camera[1] -= v

tela.fill((15, 20, 30))
desenhar_grade(tela, camera)

for i, objeto in enumerate(objetos):
    if not modo_arame:
        desenhar_malha(tela, malhas[objeto["tipo"]], objeto,
                        camera, objeto["cor"], i == selecionado)
    else:
        # Para simplificar, use a mesma funcao com preenchimento escuro.
        desenhar_malha(tela, malhas[objeto["tipo"]], objeto,
                        camera, (25,35,50), i == selecionado)
    info = f"Objeto {selecionado+1}/{len(objetos)}: {alvo['tipo']} | TAB seleciona | SPACE
arame"
    info2 = "Setas move | A/D e W/S gira | Q/E escala | J/L e I/K camera | R restaura"
    tela.blit(fonte.render(info, True, (240,240,245)), (20, 20))
    tela.blit(fonte.render(info2, True, (190,205,220)), (20, 48))
pygame.display.flip()
pygame.quit()

```

6. Controles da aplicação

Tecla	Ação
TAB	Alterna a seleção entre as instâncias.
SPACE	Alterna a visualização de superfície e modo inspirado em arame.
Setas	Movem o objeto selecionado no plano XZ.
A / D	Rotacionam o objeto em torno do eixo Y.
W / S	Rotacionam o objeto em torno do eixo X.
Q / E	Reduzem ou aumentam a escala uniforme.
J / L	Movem a câmera para a esquerda ou direita.
I / K	Movem a câmera para cima ou para baixo.
R	Restauram a câmera e a seleção iniciais.

7. Roteiro de desenvolvimento

1. Execute o código e localize as listas de vértices e faces do cubo e da pirâmide.

2. Compare o número de vértices e faces das duas malhas. Explique como cada face é formada.
3. Altere somente a posição de uma instância. Observe que a malha reutilizada não precisa ser duplicada.
4. Altere a escala e a rotação de duas instâncias do mesmo tipo. Descreva as diferenças visuais.
5. Ative o modo de arame e compare o que fica evidente nessa representação com o que fica evidente na superfície preenchida.
6. Adicione uma terceira instância de cubo usando a mesma malha, mas com outra cor, posição e escala.
7. Inclua uma indicação textual da posição, escala e rotação do objeto selecionado.
8. Teste a câmera com objetos em diferentes profundidades e explique o efeito da perspectiva.

8. Questões para discussão

- Por que separar a definição da malha dos dados de cada instância?
- Qual é a diferença entre geometria, aparência e pose?
- Como a representação em arame ajuda a inspecionar uma malha? O que ela não mostra bem?
- Que vantagem a instanciação oferece em memória, consistência e manutenção?
- Por que a ordem escala → rotação → translação foi adotada? Dê um exemplo em que trocar a ordem altera o resultado.
- Como o contorno de seleção funciona como feedback para o usuário?
- O que ocorre quando um ponto fica atrás da câmera ou muito próximo do plano de projeção?
- Como essa atividade poderia ser adaptada para seleção por clique, cursor ou traço de raio em um ambiente XR?

9. Extensões opcionais

- Adicionar um cilindro ou um prisma triangular com vértices e faces próprios.
- Implementar normais por face e variar a cor conforme a orientação da face.
- Criar uma câmera orbital com azimuth, elevação e distância ao redor do catálogo.
- Implementar seleção aproximada por clique usando caixas delimitadoras 2D.
- Criar uma hierarquia simples, como um suporte com uma pirâmide-filho.
- Adicionar rótulos com o tipo e o número de cada instância.
- Salvar e carregar a configuração dos objetos em JSON.
- Implementar recorte simples para não desenhar faces totalmente atrás da câmera.

10. Entrega esperada

Entregue o arquivo `mini_catalogo_3d.py` e um relatório curto, de uma a duas páginas, contendo:

- Descrição da solução e da organização dos dados.
- Imagem da aplicação em execução em cada modo de representação.
- Explicação de como uma mesma malha é reutilizada por diferentes instâncias.
- Exemplo que demonstre o efeito da ordem das transformações.
- Resposta às questões selecionadas e registro de pelo menos dois testes.
- Uma proposta de extensão para interação em Realidade Virtual ou Aumentada.

11. Critérios de avaliação sugeridos

Critério	Peso	Indicadores
Modelagem geométrica	20%	Vértices e faces coerentes para cubo e pirâmide.
Representação	20%	Superfícies e modo de inspeção em arame funcionando.
Instanciação	15%	Reutilização das malhas com poses e cores distintas.
Transformações e câmera	20%	Translação, rotação, escala e câmera sem falhas básicas.
Interação e feedback	15%	Seleção clara, controles responsivos e indicação visual.
Análise e criatividade	10%	Relatório, testes, respostas e extensão opcional.

12. Síntese

Nesta atividade, uma forma geométrica deixa de ser apenas uma lista de pontos: ela passa a ser uma representação organizada, reutilizável e manipulável. O exercício conecta modelagem, malhas, superfícies, instâncias, transformações, câmera, projeção e interação — fundamentos para compreender como ambientes virtuais são construídos e percebidos.